

15th Marathon of Parallel Programming WSCAD – 2020

Calebe Bianchini¹ and Marcos Amaris²

¹Mackenzie Presbyterian University

²Federal University of Pará

Rules for Remote Contest

For all problems, read carefully the input and output session. For all problems, a sequential implementation is given, and it is against the output of those implementations that the output of your programs will be compared to decide if your implementation is correct. You can modify the program in any way you see fit, except when the problem description states otherwise. You must upload a compressed file (*zip*) with your source code, the *Makefile* and an execution script. The script must have the name of the problem. You can submit as many solutions to a problem as you want. Only the last submission will be considered. The *Makefile* must have the rule `all`, which will be used to compile your source code. The execution script runs your solution the way you design it – it will be inspected not to corrupt the target machine.

The execution time of your program will be measured running it with `time` program and taking the real CPU time given. Each program will be executed at least three times with the same input and the mean time will be taken into account. The sequential program given will be measured the same way. You will earn points in each problem, corresponding to the division of the sequential time by the time of your program (speedup). The team with the most points at the end of the marathon will be declared the winner.

This problem set contains 6 problems; pages are numbered from 1 to 8.

General Information

Compilation

You should use **CC** or **CXX** inside your *Makefile*. Be careful when redefining them! There is a simple *Makefile* inside your problem package that you can modify. Example:

```
FLAGS=-O3
EXEC=sum
CXX=icpc

all: $(EXEC)

$(EXEC):
    $(CXX) $(FLAGS) $(EXEC).cpp -c -o $(EXEC).o
    $(CXX) $(FLAGS) $(EXEC).o -o $(EXEC)
```

Each judge machine has its group of compilers. See them below and choose well when writing your *Makefile*. The compiler that is tagged as *default* is predefined in **CC** and **CXX** variables.

machine	compiler	variables
host / tsubasa	GCC 4.8.5 20150623 (<i>default</i>)	CC=gcc CXX=g++
	GCC 10.2.0	CC=gcc-10.2 CXX=g++-10.2
	NEC Compiler 3.0.8	CC=ncc CXX=nc++
MPI	GCC 8.3.0 (<i>default</i>)	CC=gcc CXX=g++
	GCC 10.2.0	CC=gcc-10.2 CXX=g++-10.2
	Open MPI 3.1.3	CC=mpicc CXX=mpic++
knl	GCC 8.3.0 (<i>default</i>)	CC=gcc CXX=g++
	GCC 10.2.0	CC=gcc-10.2 CXX=g++-10.2
	Intel C/C++ Compiler 19.1.0.166	CC=icc CXX=icpc
gpu	GCC 8.3.0 (<i>default</i>)	CC=gcc CXX=g++
	GCC 10.2.0	CC=gcc-10.2 CXX=g++-10.2
	Cuda 10.2, V10.2.89, Driver Ver: 440.33.01	CC=nvcc CXX=nvcc

Submitting

General information

You must have an execution script that has the same name of the problem. This script runs your solution the way you design it. There is a simple script inside your problem package that should be modified. Example:

```
#!/bin/bash
# This script runs a generic Problem A
# Using 32 threads and OpenMP
export OMP_NUM_THREADS=32
OMP_NUM_THREADS=32 ./sum
```

Submitting MPI

If you are planning to submit an MPI solution, you should compile using *mpicc/mpic++*. The script must call *mpirun/mpiexec* with the correct number of processes (max: 4). It must use a file called *machines* that are generated by the *auto-judge* system - **do not** create it.

```
#!/bin/bash
# This script runs a generic Problem A
# Using MPI in the entire cluster (4 nodes)
# 'machines' file describes the nodes
mpirun -np 4 -machinefile machines ./sum
```

Comparing times & results

In your personal machine, measure the execution time of your solution using *time* program. Add input/output redirection when collecting time. Use *diff* program to compare the original and your solution results. Example:

```
$ time -p ./A < original_input.txt > my_output.txt
real 4.94
user 0.08
sys 1.56

$ diff my_output.txt original_output.txt
```

Do not measure time and **do not** add input/output redirection when submitting your solution - the *auto-judge* system is prepared to collect your time and compare the results.

Problem E

Yet More Primes

Prime test is a common task in many applications, specially for security, cryptography, and other interest areas. There are a number of techniques used to verify if a number is prime, and the common problem is that all techniques consumes lots of time.

The problem is easy to solve, giving the number to be tested and time enough to calculate. But a weird scientist took our numbers, and cut them in two halves, so we have first to re-arrange the numbers, looking for the primes. In this case, we have the certainty that all the numbers we have are primes, but the problem is how to re-order them

Input

In the first line, there is the number P of prime numbers ($1 \leq P \leq 1000$). Then, $2P$ lines with parts of the numbers to be tested, with no order. Prime numbers in this problem are taken as strings, so first half and second half simply means an arbitrary cut of such string. For instance, prime number 504155039 can be cut in 5041 and 55039, or 504155 and 039

Note that second half numbers could begin with a lead zero, so you must take it into account. Of course, this doesn't happen with the first half.

The input must be read from the standard input.

Output

You have to print the list of the P prime numbers, in ascendant order.

The output must be written to the standard output.

Example

Input	Output
4	1000213
50415	5041501
100	5575001
5041	504155039
55750	504155713
5039	
01	
0213	
55713	